

ARCHITECTURE CLASSIFICATION

# Linux kernel

torvalds/linux @ 2c7c88a412aa

SUBJECT

Implemented software system

FRAMEWORK

arc4d3 Scope-A · TBox 2.0.5-scope-a

SCOPE

20 core dimensions · 5 categories · 1 overlay

EVIDENCE

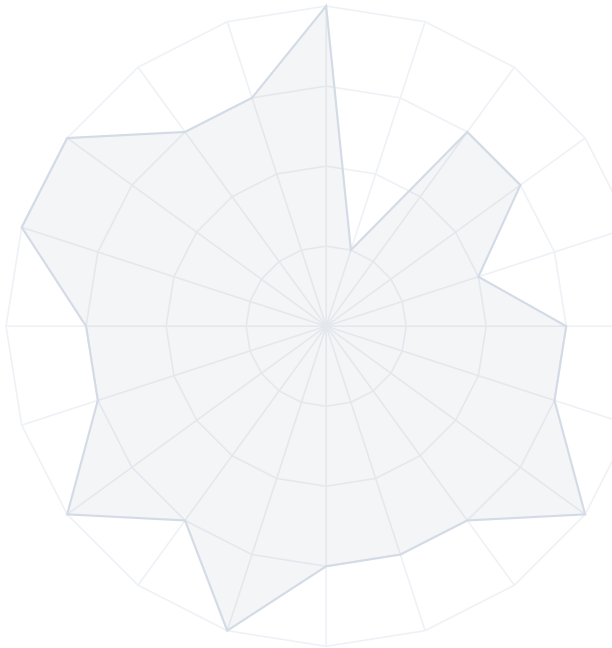
codebase-mapper 0.5.0 · 94,841 files

AUTHOR

Claude Fable 5 (via Claude Code)

GENERATED

2026-07-09



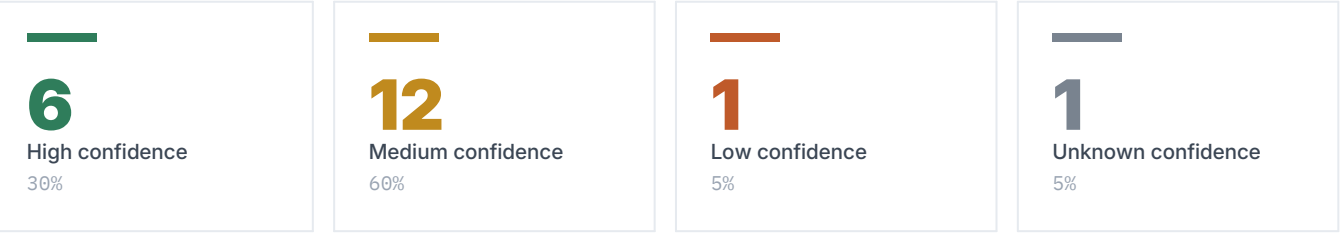
● UNVERIFIED — PENDING SHACL VALIDATION

LLM-authored per PALS's Law. Not an authoritative source until Step-2 SHACL validation exits 0.

EXECUTIVE SUMMARY

# Classification overview

Twenty independently variable core dimensions of the Linux kernel, each assigned a value and an evidential confidence level from a single codebase-mapper bundle.

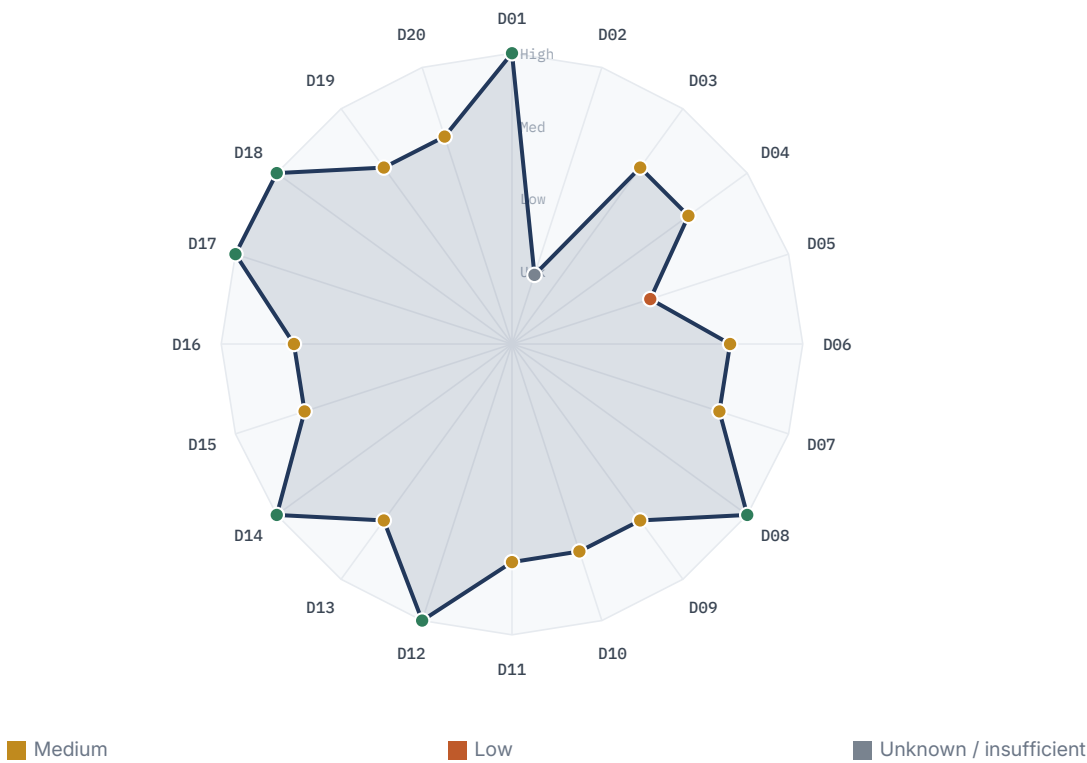


## Confidence distribution



## Confidence profile across all dimensions

Distance from centre encodes evidential confidence, not design quality — a spoke drawn toward the centre marks a dimension whose structure could not be firmly established from this bundle (D02, D05).



DETAILED FINDINGS

Dimension-by-dimension classification

Grouped by category. ● marks the dominant value where a dimension is layered across several values; ▲ flags a dimension whose conclusion is bounded by a recorded evidence risk.

| STRUCTURAL      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 5 classifications |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| D01             | <div><p><b>Decomposition model</b></p><p>scope: system</p><p>by-subsystem (functional domain partitions)</p><p>Top-level tree partitions by functional domain, not by layer or product feature: list_files totals — drivers/=37,500, fs/=2,369, kernel/=722, security/=308, tools/testing/selftests/=5,161 of 94,841 files (orient_bundle). Domain concepts dominate: repository_summary key_concepts device file_count=23,219, irq=5,201, probe=11,233.</p></div>                                                                                                   | HIGH              |
| D02             | <div><p><b>Dependency topology</b></p><p>scope: system risk flagged</p><p>no classification value — insufficient evidence</p><p>Insufficient evidence: the bundle resolves only ~22% of extracted includes to intra-repo edges (run_manifest counts.import_edges=91,736 vs ast_coverage imports_extracted=407,936) because angle-bracket includes are dropped; imported_by(path=include/linux/fs.h) returns []. Would be settled by regenerating the bundle with the angle-include-resolving extractor and running cycle detection on the completed graph.</p></div> | UNKNOWN           |
| D03             | <div><p><b>Connector and interaction model</b></p><p>scope: system</p><p>direct call connector</p><p>All 91,736 measured intra-repo dependencies are direct file-level includes feeding function calls (repository_summary dependency_summary internal_imports=91736); no broker/queue/stream middleware appears among external dependencies (external_imports=9, all Python doc tooling; pins_dependency=0, per run_manifest counts). In-kernel signalling (IRQs, notifiers) exists but the dominant connector between subsystems is the direct call.</p></div>     | MEDIUM            |
| D08             | <div><p><b>Codebase physical organization</b></p><p>scope: system</p><p>monorepo organization</p><p>Single repository at commit 2c7c88a412aa holds all 94,841 files spanning core, drivers, tooling (tools/=perf, selftests), and documentation (orient_bundle bundle metadata; repository_summary total_files=94841).</p></div>                                                                                                                                                                                                                                     | HIGH              |
| D10             | <div><p><b>Composition and reuse model</b></p><p>scope: system</p><p>● plugin composition / library composition</p><p>The ops-struct (function-pointer vtable) plugin pattern is pervasive: concept_detail(name=ops) frequency=21,157 across file_count=11,295, co-occurring most with device, probe, driver, id — the driver-model registration idiom. Loadable-module infrastructure (Kbuild tristate machinery, root Kconfig) provides the composition boundary; shared helpers compose as libraries (lib/, tools/lib/).</p></div>                                | MEDIUM            |
| BOUNDARY & INFO |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 5 classifications |

**D04 Interface and contract model**

MEDIUM

scope: system

● **code-first contract** / implicit contract

The userspace contract is expressed as code: `list_files(prefix=include/uapi)` = 960 header files forming the exported ABI surface; no IDL/schema-first artifacts appear in `files_by_type` (no proto/openapi types detected). Internal kernel interfaces are implicit header contracts with no machine-checked compatibility layer: central files are bare struct-exposing headers, e.g. `fs/xfs/libxfs/xfs_format.h` imported\_by 188 files (repository\_summary central\_files).

**D05 State and data ownership model**

LOW

scope: system

shared-address-space ownership

Structural inference only: all subsystems compile into one image sharing kernel address space; cross-subsystem state types are exposed via shared headers (central\_files are cross-imported state-bearing headers, e.g. `drivers/gpu/drm/amd/amdgpu/amdgpu.h` import\_degree=460). Runtime ownership discipline (locking domains, per-CPU data) is not observable from this bundle — single-source inference, hence Low.

**D09 Abstraction and encapsulation model**

MEDIUM

scope: system

● **concretion-exposed encapsulation** / opaque module encapsulation

The most-imported artifacts are concrete-struct-exposing headers: `fs/xfs/libxfs/xfs_format.h` (on-disk format structs, imported\_by 188), `drivers/gpu/drm/amd/amdgpu/amdgpu.h` (device structs, import\_degree 460) per repository\_summary central\_files. C offers no compiler-enforced module privacy; opaque-pointer discipline exists in places but the exposed-concretion form dominates the measured import spine.

**D17 Type and data discipline**

HIGH

scope: system

static type discipline

Statically compiled languages carry the system: `c=50,012 + cpp=246 + rust=473` source files vs 422 python and 1,132 shell confined to tooling directories (run\_manifest files\_by\_language; entry\_points are all tools/ scripts per repository\_summary). Rust region adds `#[repr(C)] x104` and `#[repr(transparent)] x130` layout-discipline attributes (rust\_attribute\_distribution).

**D19 Security and trust-boundary model**

MEDIUM

scope: system

kernel/user privilege trust boundary

The privilege boundary is structurally visible from two sides: the exported syscall/ABI surface is quarantined in `include/uapi` (960 files, list\_files) and a pluggable in-kernel policy layer lives under `security/` (308 files incl. apparmor policy machinery, list\_files prefix=security). Enforcement semantics (capability checks, LSM hook coverage) are not verifiable from bundle structure alone — hence Medium, not High.

## RUNTIME &amp; OPS

3 classifications

**D06 Runtime execution model**

MEDIUM

scope: system

interrupt-driven preemptive runtime

Interrupt machinery pervades the codebase: repository\_summary key\_concepts irq frequency=33,296 across file\_count=5,201; the scheduler and preemption core live under kernel/ (722 files, incl. kernel/trace/trace.h per list\_files prefix=kernel). The kernel is itself the runtime rather than a client of one — canonical request/event-loop runtime values do not apply.

**D16 Error-handling and recovery model**

MEDIUM

scope: system

result/either error model

Errors are values (negative errno returns), not exceptions: semantic\_neighbors(q='goto cleanup error handling return -EINVAL errno') surfaces the pattern across both language regions — C cleanup functions (tools/testing/selftests/arm64/fp/fp-stress.c child\_cleanup; drivers/net/wireless/ath/ath12k/dp\_rx.c) and the Rust bridge rust/kernel/alloc/kvec/errors.rs Error::from plus drivers/android/binder/trace.rs to\_errno, which exist precisely to map Rust Result errors onto kernel errno.

**D18 Operational observability and control model**

HIGH

scope: system

logs/metrics/traces observability

Tracing machinery sits on the measured import spine: include/trace/define\_trace.h is a top-10 central file (imported\_by 193, repository\_summary central\_files) — every importer defines tracepoints; kernel/trace/trace.h is an 81,864-byte core (list\_files prefix=kernel); the perf toolchain ships in-tree (tools/perf record/report scripts are the detected entry points).

LIFECYCLE & EVOLUTION

5 classifications

**D07 Deployment and distribution model**

MEDIUM

scope: system

monolithic image with loadable modules

Build system produces one bootable image plus loadable modules: 3,208 build\_script files (per-directory Makefile/Kbuild, run\_manifest files\_by\_type) orchestrated by the root Kconfig (file\_detail path=Kconfig, concept 'kconfig'). Container/serverless/package-registry deployment forms are absent from the tree.

**D11 Change and evolution model**

MEDIUM

scope: system

stable external boundary, unstable internals

Two independent structural sources: (1) a physically segregated frozen contract surface — list\_files(prefix=include/uapi)=960 headers kept apart from mutable internal headers; (2) an in-tree change-process corpus — list\_files(prefix=Documentation/process)=56 documents (process docs are claims, but the uapi split is enforced structure). Kernel versioning is not semver; internal interfaces evolve freely behind the stable boundary.

D12 Variability and configuration-space model

HIGH

scope: system

product-line variability

Kconfig is executable compile-time product-line machinery observed directly in the bundle: root Kconfig present (file\_detail path=Kconfig, lexicalizes concept 'kconfig'), 7,266 configuration-type files repo-wide (run\_manifest files\_by\_type.configuration), driving per-option inclusion/exclusion of the 3,208 Kbuild build scripts. Each kernel config is a distinct product variant built from the same tree.

D13 Verification and assurance model

MEDIUM

scope: system risk flagged

unit/integration/e2e verification portfolio

A layered test portfolio ships in-tree: list\_files(prefix=tools/testing/selftests)=5,161 files (kselftest end-to-end suites incl. bpf, arm64, hid per central\_files and semantic\_neighbors hits), 156 Rust #[test] items (repository\_summary rust\_attribute\_distribution), rust\_files\_with\_inline\_tests=28 (run\_manifest). The bundle's own cbm:tests edge count (139) drastically undercounts this — see attached risk.

D20 Build and delivery model

MEDIUM

scope: system

Kconfig-driven make/Kbuild build

Build side is directly observed: 3,208 Makefile/Kbuild build scripts (run\_manifest files\_by\_type.build\_script) + root Kconfig (file\_detail) form the declarative per-option build. Delivery/release process is NOT observable in-tree (no CI pipeline configs detected; maintainer pull-request flow lives outside the repo; bundle's git history is depth-1) — the delivery half of this dimension would be settled by external release infrastructure, so the classification covers the build model only.

CODE EXPRESSION

3 classifications

D14 Programming paradigm model

HIGH

scope: system risk flagged

● procedural paradigm / object-oriented paradigm

Procedural C dominates: files\_by\_language c=50,012 of 65,340 source files (run\_manifest) — an undercount, since 13,537 headers mislabeled objective-c are C (see attached risk), raising the true C share further. Object-orientation appears as the ops-struct idiom in C (concept\_detail ops file\_count=11,295), not as a class-based language.

D14 Programming paradigm model

MEDIUM

scope: rust/ (Rust-for-Linux)

object-oriented paradigm / functional paradigm

Trait/impl-structured Rust region: list\_files(prefix=rust/kernel)=161 files (473 Rust files repo-wide per files\_by\_language); rust\_items sidecar counts 1,447 impl blocks, 41 traits, 415 structs, 1,935 methods (run\_manifest rust\_items\_sidecar.by\_kind); Result-typed error composition per rust/kernel/alloc/kvec/errors.rs Error::from (semantic\_neighbors hit).

**D15 Implementation idiom and style model****MEDIUM**

scope: system

idiomatic implementation

Style enforcement tooling ships in-repo and is itself substantial: `file_detail(path=scripts/checkpatch.pl)` — 245,328 bytes, concepts [checkpatch, script]; Documentation/process (56 files) carries the coding-style corpus. The single `ops-struct/driver-probe` idiom recurring across 11,295 files (`concept_detail ops`) indicates one enforced house style rather than per-module dialects.

## Risk findings

Evidence-quality caveats that bound every conclusion above. Each represents a way the bundle understates the real system.

1

Dependency-topology evidence is undersampled: `run_manifest` counts `import_edges=91,736` while `ast_coverage` totals `imports_extracted=407,936` (~22% resolution) because angle-bracket includes are not resolved intra-repo by the extractor; `imported_by(path=include/linux/fs.h)` returns `[]` despite `fs.h` being among the kernel's most-included headers. Any coupling or blast-radius conclusion drawn from this bundle understates reality.

2

13,537 C headers are mislabeled `language=objective-c` in this bundle (a single Octave `.m` file triggers a repo-wide `.h` retag; root-caused in extractor). Language census understates C and those headers were parsed with the wrong grammar and excluded from L4 enrichment scope. Corrected extractor exists but this bundle predates it.

3

Verification signal is under-measured: `files_by_type` counts only 626 `test_code` files and `tests_edges=139`, while `list_files(prefix=tools/testing/selftests)` alone returns 5,161 files. The stem-matching test heuristic does not recognize `kselftest` naming; assurance conclusions from `cbm:tests` edges are unreliable for this system.

### O01 — Quality-attribute prioritization

Assessment overlay · reads across the core dimensions; not an orthogonal peer dimension.

O01 overlay (revealed priority ordering, emergent-only — no in-tree ADRs; Documentation/process is process, not priority declarations): hardware breadth (`drivers/=37,500` files, 40% of tree) and backward compatibility of the userspace boundary (segregated `include/uapi`, 960 frozen headers) rank above internal-interface stability (no compatibility layer for internal headers, D11) and above measured in-tree test density (626 typed test files vs 65,340 source). Structural reality and stated priorities could not be compared: the tree states none. Explicitness: emergent-only.